Test - Sesión 3

Test de la sesión 3

1. **En la práctica 1, ¿quién escribe el veredicto «discrepancia» del ticket de abril?**

a) El modelo de visión, al comparar la imagen con lo que recuerda de caja.

b) El código, que compara los campos extraídos con la consulta a caja y el ticket consigo mismo.

c) Pydantic, al validar el JSON que devuelve el modelo.

d) Ollama, a partir de la legibilidad que declara el modelo.

2. **Con la imagen muy borrosa, el modelo devolvió un JSON que Pydantic aceptó, con el pedido «8-MAD-MAL-000-0005» y legibilidad «buena». ¿Qué ocurre?**

a) Se acepta: si pasa la validación de Pydantic, el contenido es correcto.

b) Se acepta, porque el propio modelo dice que la legibilidad es buena.

c) El código lo caza por formato y coherencia (ese pedido no puede existir y el ticket no cuadra consigo mismo) y da «lectura dudosa».

d) El código da «discrepancia» y avisa de un posible fraude en la tienda.

3. **El ticket de abril se lee bien: total impreso, 231,99 €. En caja, ese pedido suma 211,99 €. ¿Qué podemos afirmar?**

a) Que hay una discrepancia de 20 € que hay que revisar; no demuestra fraude ni dice quién se equivocó.

b) Que es un fraude de la tienda: el cliente pagó 20 € de más.

c) Que la diferencia la explican los dos abonos posteriores de ese pedido.

d) Que el modelo leyó mal el total, porque no coincide con caja.

4. **En la práctica 2, el OCR (PP-OCRv6) lee bien cada número del parte de incidencia escaneado, con confianza ≥ 0,97. ¿Qué le falta para que sirva a un RAG?**

a) La estructura: devuelve líneas sueltas («1», «6», «6», «0»…) y no sabe que forman una tabla ni de qué columna es cada número. El parser de documentos (PaddleOCR-VL) la reconstruye.

b) Nada: con una confianza tan alta, el texto ya está comprobado.

c) Traducirlo al inglés, porque los modelos de embeddings no entienden un escaneado en español.

d) Pasarlo por el VLM de los tickets para que corrija la ortografía.

5. **Abrimos http://<equipo>:8000/docs. ¿Qué es esa página?**

a) Una copia de la documentación oficial de Ollama.

b) La interfaz de chat de Open WebUI.

c) El contrato de la API (OpenAPI), generado por FastAPI a partir de los modelos de Pydantic, con «Try it out» para probar cada operación.

d) Un manual escrito a mano que hay que actualizar cada vez que cambia el código.

6. **Enviamos {"consulta": "pedido", "pedido_id": "1; DROP TABLE ventas"} a /datos/consulta. ¿Qué pasa y por qué?**

a) Se borra la tabla de ventas: la API ejecuta lo que recibe.

b) Un 422 con PARAMETRO_NO_VALIDO: el SQL lo escribimos nosotros, el valor va como parámetro y antes se comprueba su formato. La base no se toca.

c) El modelo detecta la intención maliciosa y se niega a responder.

d) Un 200 con una tabla vacía, porque ese pedido no existe.

7. **En el Jetson, el OCR de PaddleOCR (PP-OCRv6) tardaba unos 40 s por página (y en** modo 30 W ni arrancaba); ahora tarda unos 2 s. ¿Qué cambió?

a) Se redujo cada página a 300 px para que el modelo tenga menos que leer.

b) Se cambió PaddleOCR por el VLM general de Ollama, que es más rápido.

c) Se activó una caché: las páginas ya no se vuelven a leer.

d) El paquete de PaddlePaddle para ARM solo usa la CPU; PaddleOCR ejecuta ahora sus modelos con PyTorch (el motor «transformers»), que en el Jetson sí usa l**a GPU.**

8. **En Open WebUI elegimos «DropLab Analista» y adjuntamos el ticket. ¿Dónde está la lógica que lee la imagen, consulta los datos y busca en los documentos?**

a) En la API y el flujo del backend; la Pipe solo saca la pregunta y la imagen, llama a la API y da formato a la respuesta.

b) En la Pipe, que contiene todo el código del asistente.

c) En el RAG propio de Open WebUI, donde subimos los documentos de la empresa.

d) En el navegador de quien hace la pregunta.

9. **En el nodo «planificar», el modelo propone las tareas (imagen, cifras, documentos). ¿Qué hace el código con esa propuesta?**

a) La ejecuta tal cual: el modelo decide libremente qué herramientas usar.

b) La ignora y usa siempre las mismas reglas fijas.

c) La valida con Pydantic (opciones cerradas), la corrige si hace falta (si hay imagen, se lee) y, si falla, usa reglas.

d) Se la pasa a otro modelo más grande para que la revise.

10. **Con --sin-framework, el mismo proceso funciona con un while y unos if, y da el mismo resultado. Entonces, ¿qué aporta LangGraph?**

a) Respuestas más correctas, porque razona mejor que Python.

b) Pasos y conexiones declarados que se pueden dibujar, un estado con tipos, el progreso paso a paso y, si el proceso crece, pausar o pedir aprobación humana.

c) Que el asistente funcione sin Ollama.

d) Nada: LangGraph solo sirve si se usa también LangChain.

11. PaddleOCR-VL convierte el gráfico G1 (pág. 4 del informe de junio) en una tabla con sus 12 porcentajes. ¿Qué dice la ficha de esa página y por qué?

a) «Comprobado»: PaddleOCR-VL está especializado en documentos y no se equivoca.

b) Nada especial: un porcentaje leído de una imagen vale lo mismo que uno escrito en el texto.

c) Que hay que quitarla del índice, porque un RAG no puede usar gráficos.

d) «Sin revisar · … solo en la imagen (SIN COMPROBAR)»: esos valores no están en el texto del PDF, así que no se pueden comprobar con él.

12. El informe aprobado de junio (LOG-2026-06 v1.1) lleva en un anexo el borrador v0.9, con el mismo código. Con la regla de la sesión 2 («sustituye a» + código), ¿qué pasaba y cómo se resolvió?

a) El código no podía distinguir las dos versiones (mismo código); ahora la versión forma parte de la clave y el anexo queda SUSTITUIDO por v1.1.

b) Los embeddings separaban bien las dos versiones, así que no hizo falta hacer nada.

c) Se borró el anexo del PDF para que el RAG no lo encontrara.

d) El LLM decide en cada respuesta cuál de las dos versiones es la buena.

13. En el examen de recuperación (14_evaluar_rag.py, o 06_evaluar.py --parte recuperacion) falla una pregunta. ¿Qué tiene sentido revisar?

a) El modelo de texto: hay que cambiarlo por uno más grande.

b) El índice, el troceado, el modelo de embeddings o la búsqueda: esa parte no usa el modelo de texto.

c) El modelo de visión, que es el más lento.

d) Nada: si la respuesta final es buena, la recuperación da igual.

14. En el gráfico G1, la peor semana de Malasaña es la del 08 al 14 de junio (68 %). La caja húmeda del albarán A-0617-04 llegó el 17/06. ¿Qué se puede afirmar?

a) Que la caja húmeda causó el valle del 68 %.

b) Que el valle demuestra que la reposición fracasó y hay que cambiar de transportista.

c) Que la caja húmeda no explica ese valle (llegó la semana siguiente) y que, además, G1 no prueba causas: describe incidencias.

d) Que el gráfico está mal leído, porque las fechas no coinciden.

15. En una prueba, el asistente respondió «66 pares… [D4]», y D4 era la pág. 8 del informe, donde no aparece el 66 (está en la T1, pág. 2). El verificador del flujo no avisó. ¿Por qué?

a) Porque el 66 es una cifra pequeña y el verificador solo mira cifras de más de tres dígitos.

b) Porque el verificador comprueba el significado de cada frase y esta es correcta.

c) Porque las páginas leídas con OCR no se verifican.

d) Porque comprueba que cada cifra esté en alguna de las evidencias, no en la que se cita: hay que leer la fuente citada.